

Тестовое задание Вариант №1

Описание программы

Данная программа осуществляет сортировку объектов по марке цвета. Допустимые значения которой (К, З, С). Порядок сортировки задается правилом по типу К>З>С. Для сортировки данных по маркам используются объекты с атрибутом mark, по типу класса Item.

class Item:

```
def __init__(self, mark, value):
```

```
    self.mark = mark
```

```
    self.value = value
```

Целью данного подхода является способность сортировать различные объекты, ведь в value может храниться число, строка, словарь или объект пользовательского типа данных. Также пользователь может добавлять атрибуты. В unit тестах (TC-UNIT-POS-06) показана возможность сортировки разнородных данных. В консольном приложении данные поступают в виде строки, каждая буква которой приводится к объекту Item (это выполняет функция parse_user_input из console_app.py) .

Варианты Запуска

- 1) Запуск консольного приложения - файл console_app.py:
- 2) Запуск Unit Тестов - файл tests_unit.py:
- 3) Запуск теста производительности - файл performance_test.py
- 4) Для использования в своих проектах используйте файл sort_by_color_mark.py:

Условные обозначения в идентификаторах тестов

CON-POS	Позитивный кейс для консольного приложения
CON-ERR	Негативный кейс для консольного приложения
CON-EXIT	Выход из программы
CON-UI	Удобство пользователя
PERF	Производительность
UNIT-POS	Позитивный кейс для unit тестов
UNIT-ERR	Негативный кейс для unit тестов

Перечень Тест-кейсов для ручного тестирования

Позитивные тесты для консольного приложения

1.1 Название: Успешный ввод с первого раза (правило $K < 3 < C$)

1.2 ID: TC-CON-POS-01

1.3 Тип теста: Позитивный

1.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

1.5 Шаги

1.5.1 При запросе правила ввести $K < 3 < C$

1.5.2 При запросе последовательности ввести KC3ZKCK

1.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

1.6 Ожидаемый результат

1.6.1 После шага 1.5.2:

```
Исходная последовательность (индекс:метка):  
0:K, 1:C, 2:3, 3:3, 4:K, 5:C, 6:K  
  
Результат сортировки (метка(исходный индекс)):  
K(0), K(4), K(6), 3(2), 3(3), C(1), C(5)  
  
Кратко (только метки): KKK33CC  
  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

1.6.2 После шага 1.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

2.1 Название: Успешный ввод с первого раза (правило с ' $>$ ' $C < 3 < K$, эквивалентно $K < 3 < C$)

2.2 ID: TC-CON-POS-02

2.3 Тип теста: Позитивный

2.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

2.5 Шаги

2.5.1 При запросе правила ввести $C > 3 > K$

2.5.2 При запросе последовательности ввести KC33KCK

2.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

2.6 Ожидаемый результат

2.6.1 После шага 2.5.2:

```
Исходная последовательность (индекс:метка):  
0:K, 1:C, 2:3, 3:3, 4:K, 5:C, 6:K  
  
Результат сортировки (метка(исходный индекс)):  
K(0), K(4), K(6), 3(2), 3(3), C(1), C(5)  
  
Кратко (только метки): KKK33CC  
  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

2.6.2 После шага 2.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

3.1 Название: Правило с пробелами

3.2 ID: TC-CON-POS-03

3.3 Тип теста: Позитивный

3.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

3.5 Шаги

3.5.1 При запросе правила ввести $K < 3 < C$

3.5.2 При запросе последовательности ввести KC33KCK

3.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

3.6 Ожидаемый результат

3.6.1 После шага 3.5.2:

```
Исходная последовательность (индекс:метка):  
0:K, 1:C, 2:3, 3:3, 4:K, 5:C, 6:K  
  
Результат сортировки (метка(исходный индекс)):  
K(0), K(4), K(6), 3(2), 3(3), C(1), C(5)  
  
Кратко (только метки): KKK33CC  
  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

3.6.2 После шага 3.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

4.1 Название: Последовательность без цвета 3

4.2 ID: TC-CON-POS-04

4.3 Тип теста: Позитивный

4.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

4.5 Шаги

4.5.1 При запросе правила ввести K<3<C

4.5.2 При запросе последовательности ввести KCKC

4.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

4.6 Ожидаемый результат

4.6.1 После шага 4.5.2:

```
Введите правило (например, K<3<C или C>3>K):
K<3<C

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKCK

Исходная последовательность (индекс:метка):
0:K, 1:C, 2:K, 3:C

Результат сортировки (метка(исходный индекс)):
K(0), K(2), C(1), C(3)

Кратко (только метки): KKCC

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

4.6.2 После шага 4.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

5.1 Название: Один цвет в последовательности (все K)

5.2 ID: TC-CON-POS-05

5.3 Тип теста: Позитивный

5.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

5.5 Шаги

5.5.1 При запросе правила ввести K<3<C

5.5.2 При запросе последовательности ввести KKK

5.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

5.6 Ожидаемый результат

5.6.1 После шага 5.5.2:

```
Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKK

Исходная последовательность (индекс:метка):
0:K, 1:K, 2:K

Результат сортировки (метка(исходный индекс)):
K(0), K(1), K(2)

Кратко (только метки): KKK

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

5.6.2 После шага 5.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

6.1 Название: Один символ в последовательности

6.2 ID: TC-CON-POS-06

6.3 Тип теста: Позитивный

6.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

6.5 Шаги

6.5.1 При запросе правила ввести K<3<C

6.5.2 При запросе последовательности ввести K

6.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

6.6 Ожидаемый результат

6.6.1 После шага 6.5.2:

```
Введите последовательность объектов (только K, 3, C, например: KC33KCK):  
K  
  
Исходная последовательность (индекс:метка):  
0:K  
  
Результат сортировки (метка(исходный индекс)):  
K(0)  
  
Кратко (только метки): K  
  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

6.6.2 После шага 6.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

7.1 Название: Последовательность из 1000 символов

7.2 ID: TC-CON-POS-07

7.3 Тип теста: Позитивный

7.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

7.5 Шаги

7.5.1 При запросе правила ввести К<З<С

7.5.2 Ввести длинную последовательность в 1000 символов, ее можно сгенерировать `".join(random.choice('K3C') for _ in range(1000))` .

7.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

7.6 Ожидаемый результат

Программа не вылетает при вводе 1000 символов, это главная цель проверки теста, проверка корректности сортировки 1000 символов лежит на тесте TC-PERF-01.

7.6.1 После шага 7.5.2:

```
Введите правило (например, К<3<С или С>3>К):
К<3<С

Введите последовательность объектов (только К, 3, С, например: КСЗЗКСК):
ЗКЗКККСКСЗККЗКСККЗКЗКСКККЗКСЗЗКСКСККЗЗКСЗКСКСККЗСССЗССКЗЗСССЗКЗЗЗССКЗСЗКСССССККЗЗСЗЗКСККЗЗССК

Исходная последовательность (индекс:метка):
0:3, 1:К, 2:3, 3:К, 4:К, 5:С, 6:К, 7:С, 8:3, 9:3, 10:К, 11:К, 12:3, 13:С, 14:К, 15:К, 16:К, 17:3, 18:К, 19:С, 20:К, 21:С, 22:К, 23:С, 24:К, 25:С, 26:К, 27:С, 28:К, 29:С, 30:К, 31:С, 32:К, 33:С, 34:К, 35:С, 36:К, 37:С, 38:К, 39:С, 40:К, 41:С, 42:К, 43:С, 44:К, 45:С, 46:К, 47:С, 48:К, 49:С, 50:К, 51:С, 52:К, 53:С, 54:К, 55:С, 56:К, 57:С, 58:К, 59:С, 60:К, 61:С, 62:К, 63:С, 64:К, 65:С, 66:К, 67:С, 68:К, 69:С, 70:К, 71:С, 72:К, 73:С, 74:К, 75:С, 76:К, 77:С, 78:К, 79:С, 80:К, 81:С, 82:К, 83:С, 84:К, 85:С, 86:К, 87:С, 88:К, 89:С, 90:К, 91:С, 92:К, 93:С, 94:К, 95:С, 96:К, 97:С, 98:К, 99:С, 100:К, 101:С, 102:К, 103:С, 104:К, 105:С, 106:К, 107:С, 108:К, 109:С, 110:К, 111:С, 112:К, 113:С, 114:К, 115:С, 116:К, 117:С, 118:К, 119:С, 120:К, 121:С, 122:К, 123:С, 124:К, 125:С, 126:К, 127:С, 128:К, 129:С, 130:К, 131:С, 132:К, 133:С, 134:К, 135:С, 136:К, 137:С, 138:К, 139:С, 140:К, 141:С, 142:К, 143:С, 144:К, 145:С, 146:К, 147:С, 148:К, 149:С, 150:К, 151:С, 152:К, 153:С, 154:К, 155:С, 156:К, 157:С, 158:К, 159:С, 160:К, 161:С, 162:К, 163:С, 164:К, 165:С, 166:К, 167:С, 168:К, 169:С, 170:К, 171:С, 172:К, 173:С, 174:К, 175:С, 176:К, 177:С, 178:К, 179:С, 180:К, 181:С, 182:К, 183:С, 184:К, 185:С, 186:К, 187:С, 188:К, 189:С, 190:К, 191:С, 192:К, 193:С, 194:К, 195:С, 196:К, 197:С, 198:К, 199:С, 200:К, 201:С, 202:К, 203:С, 204:К, 205:С, 206:К, 207:С, 208:К, 209:С, 210:К, 211:С, 212:К, 213:С, 214:К, 215:С, 216:К, 217:С, 218:К, 219:С, 220:К, 221:С, 222:К, 223:С, 224:К, 225:С, 226:К, 227:С, 228:К, 229:С, 230:К, 231:С, 232:К, 233:С, 234:К, 235:С, 236:К, 237:С, 238:К, 239:С, 240:К, 241:С, 242:К, 243:С, 244:К, 245:С, 246:К, 247:С, 248:К, 249:С, 250:К, 251:С, 252:К, 253:С, 254:К, 255:С, 256:К, 257:С, 258:К, 259:С, 260:К, 261:С, 262:К, 263:С, 264:К, 265:С, 266:К, 267:С, 268:К, 269:С, 270:К, 271:С, 272:К, 273:С, 274:К, 275:С, 276:К, 277:С, 278:К, 279:С, 280:К, 281:С, 282:К, 283:С, 284:К, 285:С, 286:К, 287:С, 288:К, 289:С, 290:К, 291:С, 292:К, 293:С, 294:К, 295:С, 296:К, 297:С, 298:К, 299:С, 300:К, 301:С, 302:К, 303:С, 304:К, 305:С, 306:К, 307:С, 308:К, 309:С, 310:К, 311:С, 312:К, 313:С, 314:К, 315:С, 316:К, 317:С, 318:К, 319:С, 320:К, 321:С, 322:К, 323:С, 324:К, 325:С, 326:К, 327:С, 328:К, 329:С, 330:К, 331:С, 332:К, 333:С, 334:К, 335:С, 336:К, 337:С, 338:К, 339:С, 340:К, 341:С, 342:К, 343:С, 344:К, 345:С, 346:К, 347:С, 348:К, 349:С, 350:К, 351:С, 352:К, 353:С, 354:К, 355:С, 356:К, 357:С, 358:К, 359:С, 360:К, 361:С, 362:К, 363:С, 364:К, 365:С, 366:К, 367:С, 368:К, 369:С, 370:К, 371:С, 372:К, 373:С, 374:К, 375:С, 376:К, 377:С, 378:К, 379:С, 380:К, 381:С, 382:К, 383:С, 384:К, 385:С, 386:К, 387:С, 388:К, 389:С, 390:К, 391:С, 392:К, 393:С, 394:К, 395:С, 396:К, 397:С, 398:К, 399:С, 400:К, 401:С, 402:К, 403:С, 404:К, 405:С, 406:К, 407:С, 408:К, 409:С, 410:К, 411:С, 412:К, 413:С, 414:К, 415:С, 416:К, 417:С, 418:К, 419:С, 420:К, 421:С, 422:К, 423:С, 424:К, 425:С, 426:К, 427:С, 428:К, 429:С, 430:К, 431:С, 432:К, 433:С, 434:К, 435:С, 436:К, 437:С, 438:К, 439:С, 440:К, 441:С, 442:К, 443:С, 444:К, 445:С, 446:К, 447:С, 448:К, 449:С, 450:К, 451:С, 452:К, 453:С, 454:К, 455:С, 456:К, 457:С, 458:К, 459:С, 460:К, 461:С, 462:К, 463:С, 464:К, 465:С, 466:К, 467:С, 468:К, 469:С, 470:К, 471:С, 472:К, 473:С, 474:К, 475:С, 476:К, 477:С, 478:К, 479:С, 480:К, 481:С, 482:К, 483:С, 484:К, 485:С, 486:К, 487:С, 488:К, 489:С, 490:К, 491:С, 492:К, 493:С, 494:К, 495:С, 496:К, 497:С, 498:К, 499:С, 500:К, 501:С, 502:К, 503:С, 504:К, 505:С, 506:К, 507:С, 508:К, 509:С, 510:К, 511:С, 512:К, 513:С, 514:К, 515:С, 516:К, 517:С, 518:К, 519:С, 520:К, 521:С, 522:К, 523:С, 524:К, 525:С, 526:К, 527:С, 528:К, 529:С, 530:К, 531:С, 532:К, 533:С, 534:К, 535:С, 536:К, 537:С, 538:К, 539:С, 540:К, 541:С, 542:К, 543:С, 544:К, 545:С, 546:К, 547:С, 548:К, 549:С, 550:К, 551:С, 552:К, 553:С, 554:К, 555:С, 556:К, 557:С, 558:К, 559:С, 560:К, 561:С, 562:К, 563:С, 564:К, 565:С, 566:К, 567:С, 568:К, 569:С, 570:К, 571:С, 572:К, 573:С, 574:К, 575:С, 576:К, 577:С, 578:К, 579:
```

7.6.2 После шага 7.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

8.1 Название: Повторная сортировка (цикл)

8.2 ID: TC-CON-POS-08

8.3 Тип теста: Позитивный

8.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

8.5 Шаги

8.5.1 Выполнить сортировку (например аналогично ТС-CON-POS-01)

8.5.2 На вопрос "Хотите выполнить ещё одну сортировку?" ввести у

8.5.3 Выполнить повторную сортировку (вернуться на предыдущий шаг и проверить продолжение программы с вводом (Y, YES, yes, д, Д, да, ДА))

8.5.4 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

8.6 Ожидаемый результат

8.6.1 После шага 8.5.3:

```
Хотите выполнить ещё одну сортировку? (y/n):  
y  
Введите правило (например, K<3<C или C>3>K):  
K<3<C  
Введите последовательность объектов (только K, 3, C, например: KC33KCK):  
KC33KCK  
Исходная последовательность (индекс:метка):  
0:K, 1:C, 2:3, 3:3, 4:K, 5:C, 6:K  
Результат сортировки (метка(исходный индекс)):  
K(0), K(4), K(6), 3(2), 3(3), C(1), C(5)  
Кратко (только метки): KKK33CC  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
Хотите выполнить ещё одну сортировку? (y/n):
```

8.6.2 После шага 8.5.4 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

Негативные тесты для консольного приложения

1.1 Название: Пустое правило → повторный ввод

1.2 ID: TC-CON-ERR-01

1.3 Тип теста: Негативный

1.4 Предварительное условие: Программа запущена. (О вариантах запуска смотреть README.md или раздел “варианты запуска”).

1.5 Шаги

1.5.1 При запросе правила нажать Enter

1.5.2 Увидеть сообщение "Ошибка: правило не может быть пустым"

1.5.3 Ввести корректное правило $C < Z < K$

1.5.4 Ввести корректную последовательность KKCKЗСК

1.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

1.6 Ожидаемый результат

1.6.1 После шага 1.5.4:

```
Введите правило (например, K<Z<C или C>Z>K):  
  
Ошибка: правило не может быть пустым. Попробуйте снова.  
  
Введите правило (например, K<Z<C или C>Z>K):  
C<Z<K  
  
Введите последовательность объектов (только K, Z, C, например: KCЗЗКСК):  
KKCKЗСК  
  
Исходная последовательность (индекс:метка):  
0:K, 1:K, 2:C, 3:K, 4:Z, 5:C, 6:K  
  
Результат сортировки (метка(исходный индекс)):  
C(2), C(5), Z(4), K(0), K(1), K(3), K(6)  
  
Кратко (только метки): CCЗKKKK  
  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

1.6.2 После шага 1.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

2.1 Название: Правило с недопустимой меткой (X)

2.2 ID: TC-CON-ERR-02

2.3 Тип теста: Негативный

2.4 Предварительное условие: Программа запущена. (python3 console_app.py).

2.5 Шаги

2.5.1 При запросе правила ввести $C < Z < X$

2.5.2 Увидеть сообщение " Ошибка в правиле: Недопустимая метка в правиле: 'X'. Допустимы: K, Z, C. Попробуйте снова. "

2.5.3 Ввести корректное правило $C < Z < K$

2.5.4 Ввести корректную последовательность ККСКЗСК

2.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

2.6 Ожидаемый результат

2.6.1 После шага 2.5.4:

```
Введите правило (например, К<З<С или С>З>К):
С<З<Х
Ошибка в правиле: Недопустимая метка в правиле: 'Х'. Допустимы: К, З, С. Попробуйте снова.

Введите правило (например, К<З<С или С>З>К):
С<З<К

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
ККСКЗСК

Исходная последовательность (индекс:метка):
0:К, 1:К, 2:С, 3:К, 4:З, 5:С, 6:К

Результат сортировки (метка(исходный индекс)):
С(2), С(5), З(4), К(0), К(1), К(3), К(6)

Кратко (только метки): ССЗКККК

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

2.6.2 После шага 2.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

3.1 Название: Правило без разделителя (СЗК)

3.2 ID: TC-CON-ERR-03

3.3 Тип теста: Негативный

3.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

3.5 Шаги

3.5.1 При запросе правила ввести СЗК

3.5.2 Увидеть сообщение "Ошибка в правиле: Правило должно содержать разделители '<' или '>'. Попробуйте снова."

3.5.3 Ввести корректное правило С<З<К

3.5.4 Ввести корректную последовательность ККСКЗСК

3.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

3.6 Ожидаемый результат

3.6.1 После шага 3.5.4:

```
Введите правило (например, K<3<C или C>3>K):
C3K
Ошибка в правиле: Правило должно содержать разделители '<' или '>'. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC3ZKCK):
KKCK3CK

Исходная последовательность (индекс:метка):
0:K, 1:K, 2:C, 3:K, 4:3, 5:C, 6:K

Результат сортировки (метка(исходный индекс)):
C(2), C(5), 3(4), K(0), K(1), K(3), K(6)

Кратко (только метки): CC3KKKK

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

3.6.2 После шага 3.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

4.1 Название: Правило с двумя цветами (K<3)

4.2 ID: TC-CON-ERR-04

4.3 Тип теста: Негативный

4.4 Предварительное условие: Программа запущена. (python3 console_app.py).

4.5 Шаги

4.5.1 При запросе правила ввести K<3

4.5.2 Увидеть сообщение "Ошибка в правиле: Правило должно содержать ровно 3 метки (K, 3, C), получено: ['K', '3']. Попробуйте снова."

4.5.3 Ввести корректное правило C<3<K

4.5.4 Ввести корректную последовательность KKCK3CK

4.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

4.6 Ожидаемый результат

4.6.1 После шага 4.5.4:

```
Введите правило (например, K<3<C или C>3>K):
K<3
Ошибка в правиле: Правило должно содержать ровно 3 метки (K, 3, C), получено: ['K', '3']. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC3ZKCK):
KKCK3CK

Исходная последовательность (индекс:метка):
0:K, 1:K, 2:C, 3:K, 4:3, 5:C, 6:K

Результат сортировки (метка(исходный индекс)):
C(2), C(5), 3(4), K(0), K(1), K(3), K(6)

Кратко (только метки): CC3KKKK

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

4.6.2 После шага 4.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

5.1 Название: Правило с дубликатами (K<3<K)

5.2 ID: TC-CON-ERR-05

5.3 Тип теста: Негативный

5.4 Предварительное условие: Программа запущена. (python3 console_app.py)

5.5 Шаги

5.5.1 При запросе правила ввести K<3<K

5.5.2 Увидеть сообщение " Ошибка в правиле: Правило содержит повторяющиеся метки: ['K', '3', 'K']. Попробуйте снова."

5.5.3 Ввести корректное правило C<3<K

5.5.4 Ввести корректную последовательность KKCK3CK

5.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

5.6 Ожидаемый результат

5.6.1 После шага 5.5.4:

```
Введите правило (например, K<3<C или C>3>K):
K<3<K
Ошибка в правиле: Правило содержит повторяющиеся метки: ['K', '3', 'K']. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC3ZKCK):
KKCK3CK

Исходная последовательность (индекс:метка):
0:K, 1:K, 2:C, 3:K, 4:3, 5:C, 6:K

Результат сортировки (метка(исходный индекс)):
C(2), C(5), 3(4), K(0), K(1), K(3), K(6)

Кратко (только метки): CC3KCKK

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

5.6.2 После шага 5.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

6.1 Название: Смешанные разделители в правиле (K<3>C)

6.2 ID: TC-CON-ERR-06

6.3 Тип теста: Негативный

6.4 Предварительное условие: Программа запущена. (python3 console_app.py)

6.5 Шаги

6.5.1 При запросе правила ввести K<3>C

6.5.2 Увидеть сообщение "Ошибка в правиле: Правило не должно содержать одновременно '<' и '>'. Попробуйте снова."

6.5.3 Ввести корректное правило C<3<K

6.5.4 Ввести корректную последовательность KKCK3CK

6.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

6.6 Ожидаемый результат

6.6.1 После шага 6.5.4:

```

Введите правило (например, K<3<C или C>3>K):
K<3>C
Ошибка в правиле: Правило не должно содержать одновременно '<' и '>'. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKCK3CK

Исходная последовательность (индекс:метка):
0:K, 1:K, 2:C, 3:K, 4:3, 5:C, 6:K

Результат сортировки (метка(исходный индекс)):
C(2), C(5), 3(4), K(0), K(1), K(3), K(6)

Кратко (только метки): CC3KKKK

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):

```

6.6.2 После шага 6.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

7.1 Название: Пустая последовательность

7.2 ID: TC-CON-ERR-07

7.3 Тип теста: Негативный

7.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

7.5 Шаги

7.5.1 Ввести корректное правило C<3<K

7.5.2 Ввести пустую последовательность (нажать Enter)

7.5.3 Увидеть сообщение " Ошибка: последовательность не может быть пустой. Попробуйте снова.

7.5.4 Ввести корректную последовательность KKCK3CK

7.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

7.6 Ожидаемый результат

7.6.1 После шага 7.5.4:

```
Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):  
  
Ошибка: последовательность не может быть пустой. Попробуйте снова.  
  
Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):  
ККСКЗСК  
  
Исходная последовательность (индекс:метка):  
0:К, 1:К, 2:С, 3:К, 4:З, 5:С, 6:К  
  
Результат сортировки (метка(исходный индекс)):  
С(2), С(5), З(4), К(0), К(1), К(3), К(6)  
  
Кратко (только метки): ССЗКККК  
  
Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

7.6.2 После шага 7.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

8.1 Название: Недопустимый символ в последовательности

8.2 ID: TC-CON-ERR-08

8.3 Тип теста: Негативный

8.4 Предварительное условие: Программа запущена. (python3 console_app.py)

8.5 Шаги

8.5.1 Ввести корректное правило $C < Z < K$

8.5.2 Ввести последовательность КССКЗХ

8.5.3 Увидеть сообщение "Ошибка: Недопустимый символ 'X'. Допустимы: К, З, С. Попробуйте снова."

8.5.4 Ввести корректную последовательность ККСКЗСК

8.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

8.6 Ожидаемый результат

8.6.1 После шага 8.5.4:

```
Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КССКЗХ
Ошибка: Недопустимый символ 'Х'. Допустимы: К, З, С. Попробуйте снова.

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КССКЗСК

Исходная последовательность (индекс:метка):
0:К, 1:К, 2:С, 3:К, 4:З, 5:С, 6:К

Результат сортировки (метка(исходный индекс)):
С(2), С(5), З(4), К(0), К(1), К(3), К(6)

Кратко (только метки): ССЗКККК

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

8.6.2 После шага 8.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

Тесты завершения программы для консольного приложения

1.1 Название: Выход по любому символу кроме д/да/у/yes (без учета регистра)

1.2 ID: TC-CON-EXIT-01

1.3 Тип теста: Выход из программы

1.4 Предварительное условие: Программа запущена. (**python3 console_app.py**), выполнена успешная сортировка – как это сделать можно увидеть в тесте TC-CON-POS-01

1.5 Шаги

1.5.1 Увидеть сообщение " Хотите выполнить ещё одну сортировку? (y/n):

1.5.2 Ввести символы **не** из д/да/у/yes (без учета регистра)

1.6 Ожидаемый результат

1.6.1 После шага 1.5.2:

```
Хотите выполнить ещё одну сортировку? (y/n):  
123  
До свидания!  
  
Process finished with exit code 0
```

2.1 Название: Выход по прерыванию с клавиатуры

2.2 ID: TC-CON-EXIT-02

2.3 Тип теста: Выход из программы

2.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

2.5 Шаги

2.5.1 Выполнить прерывание с клавиатуры на разных этапах работы программы (запрос правила, запрос последовательности, запрос согласия на повтор).

2.6 Ожидаемый результат

2.6.1 После шага 2.5.1:

```
Введите правило (например, К<З<С или С>З>К):  
Программа прервана пользователем  
  
Process finished with exit code 0
```

Тест удобства использования для консольного приложения

1.1 Название: Проверка сообщения об ошибке раскладки клавиатуры

1.2 ID: TC-CON-UI-01

1.3 Тип теста: Удобство использования

1.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

1.5 Шаги

1.5.1 Проверить сообщение о приглашении к вводу правила “ Введите правило (например, К<З<С или С>З>К):”.

1.5.2 При запросе правила ввести К<З<С(с английской раскладки). Проверить сообщение “Ошибка в правиле: Недопустимая метка в правиле: 'С' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.”.

1.5.3 Ввести правило К<З<С (на это раз на кириллице), проверить новое сообщение приглашение к вводу “ Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):”

1.5.4 Ввести последовательность К(с английской раскладки)ККЗСС. Увидеть сообщение “Ошибка: Недопустимый символ 'К' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.”

1.5.5 Ввести корректную последовательность на кириллице КККЗСС

1.6 Ожидаемый результат

1.6.1 После шага 1.5.5:

```
Введите правило (например, К<З<С или С>З>К):
К<З<С
Ошибка в правиле: Недопустимая метка в правиле: 'С' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.

Введите правило (например, К<З<С или С>З>К):
К<З<С

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КККЗСС
Ошибка: Недопустимый символ 'К' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КККЗСС

Исходная последовательность (индекс:метка):
0:К, 1:К, 2:К, 3:З, 4:С, 5:С

Результат сортировки (метка(исходный индекс)):
К(0), К(1), К(2), З(3), С(4), С(5)

Кратко (только метки): КККЗСС

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
|
```

Тест производительности

1.1 Название: Проверка производительности (скорость и отсутствие утечек памяти)

1.2 ID: TC-PERF-01

1.3 Тип теста: Производительность

1.4 Предварительное условие: Запущены два терминала на linux системе.

1.5 Шаги

1.5.1 В терминале №1 запустить скрипт **python3 performance_test.py**

1.5.2 В терминале №1 появятся данные о сортировке 1000 элементов и затратах времени на это (ожидаем меньше 1с). Также появится идентификатор процесса, нужно скопировать его и использовать в команде `ps -p {pid} -o pid,rss,vsz` в терминале №2. Данные будут в формате `rss` – физическая память в Кб, `vsz` – виртуальная память в Кб:

```
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ ps -p 5346 -o pid,rss,vsz
  PID  RSS    VSZ
 5346 10656 18964
```

1.5.3 В терминале №1 нажать Enter, программа выполнит сортировку 1000 элементов 10 раз сравнивая на каждой итерации расход памяти при помощи `tracemalloc`.

1.5.4 Удостовериться что программа еще не завершена и повторить команду из шага 1.5.2.

1.6 Ожидаемый результат

Терминал №1

```
1 file changed, 1 insertion(+), 1 deletion(-)
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ python3 performance_test.py
PID процесса: 2003 (для оценки памяти ps -p 2003 -o pid,rss,vsz)
Время выполнения: 0.0025 секунд
Функция выполнена.
Нажмите Enter, чтобы выполнить функцию повторно 10 раз и сравнить статистику использования памяти...

--- После итерации 1 ---
/usr/lib/python3.8/tracemalloc.py:397: size=576 B (+576 B), count=4 (+4), average=144 B
/usr/lib/python3.8/tracemalloc.py:534: size=472 B (+472 B), count=2 (+2), average=236 B
performance_test.py:44: size=424 B (+424 B), count=1 (+1), average=424 B
performance_test.py:39: size=0 B (-424 B), count=0 (-1)
/usr/lib/python3.8/random.py:200: size=356 B (+356 B), count=2 (+2), average=178 B
--- После итерации 2 ---
/usr/lib/python3.8/tracemalloc.py:209: size=848 B (+848 B), count=2 (+2), average=424 B
/usr/lib/python3.8/tracemalloc.py:165: size=816 B (+816 B), count=2 (+2), average=408 B
/usr/lib/python3.8/tracemalloc.py:472: size=744 B (+744 B), count=9 (+9), average=83 B
/usr/lib/python3.8/tracemalloc.py:519: size=568 B (+568 B), count=1 (+1), average=568 B
/usr/lib/python3.8/tracemalloc.py:131: size=548 B (+548 B), count=3 (+3), average=183 B
--- После итерации 3 ---
/usr/lib/python3.8/tracemalloc.py:125: size=1816 B (+1312 B), count=20 (+13), average=91 B
/usr/lib/python3.8/tracemalloc.py:185: size=1248 B (+864 B), count=26 (+18), average=48 B
/usr/lib/python3.8/tracemalloc.py:521: size=1264 B (+832 B), count=3 (+2), average=421 B
/usr/lib/python3.8/tracemalloc.py:472: size=1464 B (+720 B), count=27 (+18), average=54 B
/usr/lib/python3.8/tracemalloc.py:131: size=0 B (-548 B), count=0 (-3)
--- После итерации 4 ---
/usr/lib/python3.8/tracemalloc.py:125: size=72 B (-1744 B), count=1 (-19), average=72 B
/usr/lib/python3.8/tracemalloc.py:121: size=1800 B (+1296 B), count=25 (+18), average=72 B
/usr/lib/python3.8/tracemalloc.py:131: size=548 B (+548 B), count=3 (+3), average=183 B
/usr/lib/python3.8/tracemalloc.py:97: size=440 B (+440 B), count=1 (+1), average=440 B
/usr/lib/python3.8/tracemalloc.py:102: size=0 B (-440 B), count=0 (-1)
--- После итерации 5 ---
/usr/lib/python3.8/tracemalloc.py:125: size=216 B (+144 B), count=3 (+2), average=72 B
/usr/lib/python3.8/tracemalloc.py:185: size=1392 B (+96 B), count=29 (+2), average=48 B
/usr/lib/python3.8/tracemalloc.py:472: size=1584 B (+80 B), count=30 (+2), average=53 B
/usr/lib/python3.8/tracemalloc.py:122: size=56 B (-56 B), count=2 (-2), average=28 B
/usr/lib/python3.8/tracemalloc.py:476: size=308 B (+28 B), count=11 (+1), average=28 B
--- После итерации 6 ---
/usr/lib/python3.8/tracemalloc.py:121: size=2464 B (+664 B), count=29 (+4), average=85 B
/usr/lib/python3.8/tracemalloc.py:131: size=0 B (-548 B), count=0 (-3)
/usr/lib/python3.8/tracemalloc.py:102: size=440 B (+440 B), count=1 (+1), average=440 B
/usr/lib/python3.8/tracemalloc.py:97: size=0 B (-440 B), count=0 (-1)
/usr/lib/python3.8/tracemalloc.py:125: size=72 B (-144 B), count=1 (-2), average=72 B
--- После итерации 7 ---
```

Сортировка выполняется < 1с. Также видно, что нет линейного роста потребления памяти между итерациями, самые затратные вызовы системные.

Терминал №2

```

ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ ps -p 2003 -o pid,rss,vsz
  PID  RSS   VSZ
 2003 10456 18956
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ ps -p 2003 -o pid,rss,vsz
  PID  RSS   VSZ
 2003 10788 19192
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ █

```

Значения в терминале 2 должны быть схожи у разных вызовов, (отличия в пределах 10% нормально и связано с фрагментацией), это говорит об отсутствии утечек памяти.

Перечень UNIT Тест-кейсов

TC-UNIT-POS-01	Проверка всех 12 валидных правил (6 перестановок цветов с двумя вариантами знака сравнения).
TC-UNIT-POS-02	Правило с пробелами.
TC-UNIT-POS-03	Пустая последовательность при верном правиле, ожидаем возврат пустого списка.
TC-UNIT-POS-04	Отсутствует метка в последовательности
TC-UNIT-POS-05	Сохранение исходного порядка внутри группы
TC-UNIT-POS-06	Объекты с дополнительными полями и значениями разных типов (корректная сортировка при наличии дополнительных атрибутов)
TC-UNIT-NEG-01	Пустое правило, ожидаем ошибку ValueError "не может быть пустым"
TC-UNIT-NEG-02	Пустое КЗС, ожидаем ошибку ValueError "должно содержать разделители"
TC-UNIT-NEG-03	Смешанные разделители К<3>С, ожидаем ошибку ValueError "Правило не должно содержать одновременно '<' и '>'"
TC-UNIT-NEG-04	Только две метки в правиле К<3, ожидаем ошибку ValueError "Правило должно содержать ровно 3 метки"
TC-UNIT-NEG-05	4 метки в правиле К<3<С<К, ожидаем ошибку ValueError "Правило должно содержать ровно 3 метки"
TC-UNIT-NEG-06	Повторяющаяся метка в правиле К<3<К, ожидаем ошибку ValueError "Правило содержит повторяющиеся метки"
TC-UNIT-NEG-07	Недопустимый символ в правиле К<3<Х, ожидаем ошибку ValueError "Недопустимая метка в правиле: 'Х'"
TC-UNIT-NEG-08	Недопустимый латинский символ в правиле схожий с валидным К<3<С(кириллица), ожидаем ошибку ValueError "Недопустимая метка в правиле: 'С' Проверьте раскладку клавиатуры."
TC-UNIT-NEG-09	Объект с недопустимой меткой 'Х', [Item('K', 1), Item('X', 2), Item('C', 3)] ожидаем ошибку ValueError "Недопустимая метка у объекта: 'Х'"
TC-UNIT-NEG-10	Несколько объектов с недопустимыми метками [Item('P', 1), Item('Q', 2)] ожидаем ошибку ValueError "Недопустимая метка у объекта: 'Р'"
TC-UNIT-NEG-11	Объект не имеет атрибута .mark, Ожидаем AttributeError